

Saying that the plot gets gradually steeper as the list length increases is still not as precise as we want to be. Sometimes we talk colloquially about this kind of accelerating growth as “exponential.” Are we dealing with exponential growth here? Strictly speaking, there is a function called the *exponential function* defined by e^x , where e is Euler’s number, or about 2.71828. So does the number of steps required for insertion sort follow this exponential function that we could say fits the narrowest possible definition of exponential growth? We can get a clue about the answer by plotting our step curve together with an exponential growth curve, as follows. We will also import the `numpy` module in order to take the maximum and minimum of our step values.

```
import math
import numpy as np
random.seed(5040)
xs = list(range(1,100))
ys = [check_steps(x) for x in xs]
ys_exp = [math.exp(x) for x in xs]
plt.plot(xs,ys)
axes = plt.gca()
axes.set_ylim([np.min(ys),np.max(ys) + 140])
plt.plot(xs,ys_exp)
plt.title('Comparing Insertion Sort to the Exponential Function')
plt.xlabel('Number of Files in Random Cabinet')
plt.ylabel('Steps Required to Sort Cabinet')
plt.show()
```

Just like before, we define `xs` to be all the numbers between 1 and 100, and `ys` to be the number of steps required to sort randomly generated lists of each size corresponding to each `x`. We also define a variable called `ys_exp`, which is e^x for each of the values stored in `xs`. We then plot both `ys` and `ys_exp` on the same plot. The result enables us to see how the growth of the number of steps required to sort a list relates to true exponential growth.

Running this code creates the plot shown in [Figure 4-2](#).